

Master : IESE – S1

Travaux Pratiques : Microcontrôleurs ARM

TP1 : Création d'un projet de base

Objectif :

Configuration des broches GPIO pour le contrôle des LEDs et des boutons :

- Création d'un projet vide dans l'IDE pour STM32.
- Configuration des broches GPIO pour le contrôle des LEDs ou des boutons.
- Implémentation d'une application simple pour allumer et éteindre une LED à l'aide d'un bouton.
- Utilisation des interruptions externes pour détecter les pressions de bouton.

Présentation de la carte STM32F446RET6 :

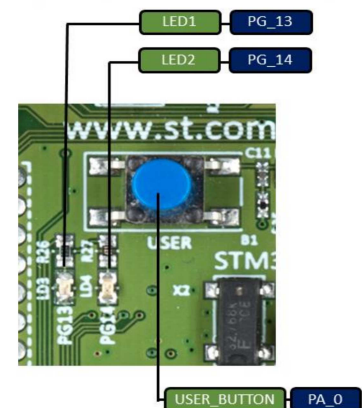
- La famille STM32F429ZI-Disc est basée sur le cœur RISC 32 bits ARM®Cortex®-M4 hautes performances avec FPU fonctionnant à une fréquence max de **180 MHz** et intégrant une unité à virgule flottante (**FPU**), une unité de protection de mémoire (**MPU**), accélérateur adaptatif en temps réel (ART), et un ensemble complet d'instructions DSP.
- Mémoire flash : 2 Mo, SRAM : 256 Ko

Caractéristiques communes :

- Microcontrôleur STM32 dans un boîtier LQFP144
- Deux LED utilisateur : **LD3** verte (connecté à l'E/S **PG13**) et **LD4** rouge (connecté à l'E/S **PG14**)
- 1 bouton-poussoir utilisateur (en **Blue**) connecté à l'E/S **PA0**.
- 1 bouton-poussoir de réinitialisation (en Noir)

Connecteurs :

- Connecteur spécial pour expériences SMPS externes
- Connecteur USB Micro-B pour la Liaison STLINK



- Connecteur de débogage avec SWD & JTAG.

System	Chrom-ART Accelerator™	Connectivity
Power supply 1.2 V internal regulator POR/PDR/PVD	ART Accelerator™	TFT LCD controller
Xtal oscillators 32 kHz + 4 ~26 MHz	180 MHz Arm® Cortex®-M4 CPU	6x SPI, 2x I²S, 3x I²C
Internal RC oscillators 32 kHz + 16 MHz	Floating Point Unit (FPU)	Camera interface
3 PLLs	Nested Vector Interrupt Controller (NVIC)	Ethernet MAC 10/100 with IEEE 1588
Clock control	JTAG/SW debug	2x CAN 2.0B
RTC/AWU	Embedded Trace Macrocell (ETM)	1x USB 2.0 OTG FS/HS
1x SysTick timer	Memory Protection Unit (MPU)	1x USB 2.0 OTG FS
2x watchdogs (independent and window)	Mutli-AHB bus matrix	1x SDMMC
82/114/140/168 I/Os	16-channel DMA	4x USART + 4 UART LIN, smartcard, IrDA, modem control
Cyclic Redundancy Check (CRC)		1x SAI (Serial audio interface)
96-bit unique ID		
Voltage scaling		
Control		Analog
2x 16-bit motor-control PWM	True random number generator (RNG)	2-channel 2x 12-bit DAC
10x 16-bit timers 2x 32-bit timers	Up to 2-Mbyte dual-bank Flash memory	3x 12-bit ADC/2.4 MSPS Up to 24 channels /7.2 MSPS
	256-Kbyte SRAM	Temperature sensor
	FMC/SRAM/NOR/NAND/ CF/SDRAM	
	80-byte + 4-Kbyte backup SRAM	
	512 OTP bytes	

Figure 1: Caractéristiques de la carte STM32F446RET6

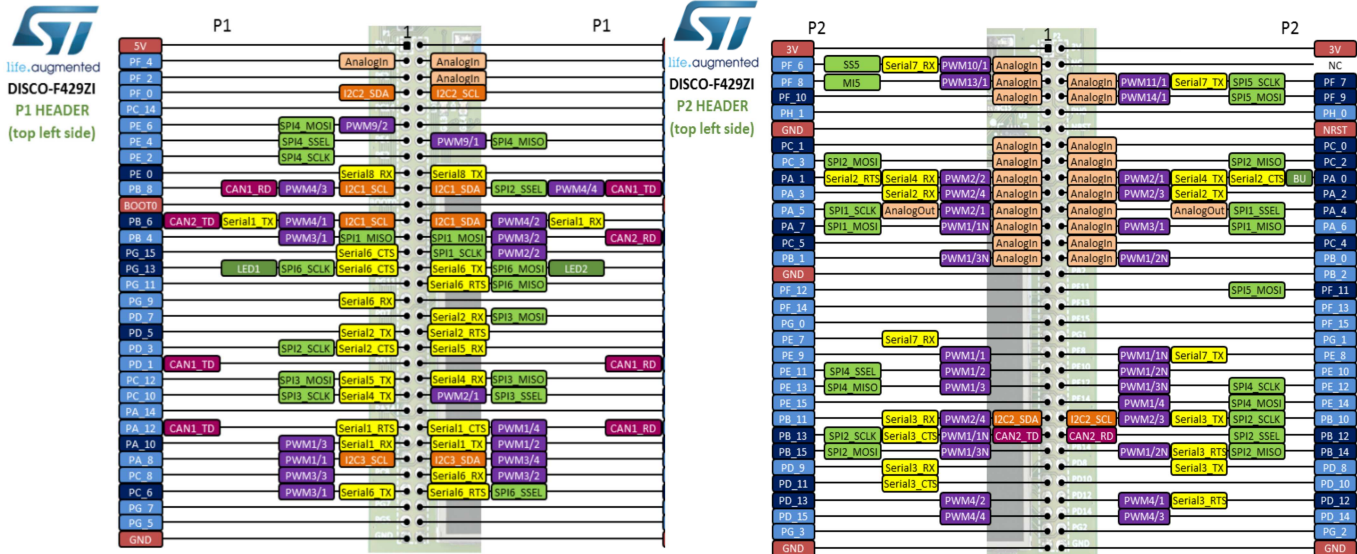


Figure 2: Pinout de STM32F429ZI

I. Environnement de développement

La programmation d'une carte STM32 s'effectue à l'aide de l'écosystème STM32Cube qui comprend principalement :



STM32CubeMX : un outil graphique permettant de configurer les broches, les horloges et les périphériques, puis de générer automatiquement le code d'initialisation.



STM32CubeIDE : un environnement de développement intégré (IDE) basé sur Eclipse, incluant l'éditeur de code, le compilateur, le débogueur et l'interface de programmation (ST-Link).

La bibliothèque HAL (Hardware Abstraction Layer) constitue le cœur de la programmation sur STM32. C'est une couche d'abstraction matérielle qui simplifie la programmation du microcontrôleur. Elle offre un ensemble de fonctions prêtes à l'emploi pour configurer et contrôler les différents périphériques sans avoir à manipuler directement les registres matériels.

Exemples de fonctions couramment utilisées :

Fonction HAL	Rôle / Description
HAL_Init()	Initialise la bibliothèque HAL et configure le système de base (horloge du SysTick, interruptions, etc.)
HAL_GPIO_WritePin(GPIOx, Pin, State)	Met une broche à l'état haut (SET) ou bas

	(RESET)
HAL_GPIO_TogglePin(GPIOx, Pin)	Inverse l'état logique d'une broche (utile pour clignoter une LED)
HAL_GPIO_ReadPin(GPIOx, Pin)	Lit l'état logique (0 ou 1) d'une broche d'entrée
HAL_GetTick(ms)	Mesure le temps écoulé en millisecondes
HAL_Delay(ms)	Génère une attente de ms millisecondes à l'aide du timer système
HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)	Fonction de rappel (callback) exécutée automatiquement lorsqu'une interruption GPIO survient
HAL_GPIO_EXTI_IRQHandler(GPIO_Pin)	Fonction appelée automatiquement par le gestionnaire d'interruptions pour traiter une ligne EXTI (généralement générée par CubeMX)

Partie 1 : Configuration des broches GPIO pour le contrôle des LEDs et des boutons

Dans cette partie, les broches GPIO seront configurées pour permettre le contrôle des LEDs et des boutons. Cela implique d'initialiser les broches GPIO appropriées en tant que sorties pour les LEDs et en tant qu'entrées pour les boutons.

1- Clignotement d'une LED

- Ouvrir STM32CubeMX.
- Sélectionner le microcontrôleur STM32 approprié : STM32F429ZI.
- Créer un nouveau projet en cliquant sur "File" > "New Project".
- Configurer les paramètres du microcontrôleur, fréquence du CPU (180 MHz), les pins GPIO, RCC, etc.
- Générer le code source du projet en cliquant sur "Project" > "Generate Code".
- Une fois le code généré, ouvrez-le dans votre environnement de développement intégré STM32CubeIDE.
- Ajouter votre propre code pour implémenter le programme de clignotement d'une LED avec une fréquence de 2 Hz.

2- Contrôle de la LED par le Button poussoir

- Configurer la carte STM32 pour contrôler une LED à l'aide d'un bouton-poussoir. Lorsque le bouton-poussoir est enfoncé, la LED s'allumera, et lorsque le bouton est relâché, la LED s'éteindra.

Dans cette partie, contrôler une LED à l'aide d'un bouton poussoir en utilisant le principe de pull-up. Cela signifie l'activation de la résistance de pull-up interne sur la broche du bouton poussoir.

Lorsque le bouton est **enfoncé**, la broche est connectée à la **masse**, ce qui provoque une lecture de niveau bas. Lorsque le bouton est **relâché**, la résistance de pull-up maintient la broche à un niveau logique **haut**.

3- Interruptions externes

a) Utilisation des interruptions externes pour détecter les pressions de bouton :

- Configurez une **interruption externe** sur la broche GPIO associée au bouton poussoir.
- Définissez une fonction de rappel (**callback**) qui sera exécutée chaque fois que l'interruption est déclenchée (c'est-à-dire lorsque le bouton est pressé).
- Dans cette fonction de rappel, effectuez les actions nécessaires en réponse à l'appui sur le bouton, telles que l'inversion de l'état de la LED.

b) Configurez la broche pour détecter à la fois les fronts montants et descendants, puis modifiez le code afin d'allumer la LED lors d'un front montant et de l'éteindre lors d'un front descendant.

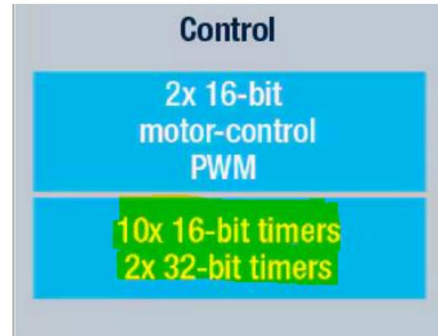
```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_0) // Si l'interruption vient du bouton PC0
    {
        HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_13); // Changer l'état de la LED
    }
}
```

Changer le programme pour qu'un seul appui sur le bouton génère un clignotement d'une fréquence de 1 Hz, deux appuis successifs génère un clignotement d'une fréquence de 2 Hz, et trois appuis éteint la LED. Un autre appui allume la LED (état normal, sans clignotement).
Un délai de 50 ms est utilisé pour éviter les faux déclenchements.

Partie 2 : Les minuteries (Timer) STM32 | Hardware Timers

Dans cette partie, nous aborderons les modules de minuterie STM32 dans les microcontrôleurs STM32. Il existe différentes minuteries matérielles dans les microcontrôleurs STM32, chacune pouvant fonctionner dans plusieurs modes et effectuer de nombreuses tâches.

Un **Timer** dans sa forme la plus élémentaire est un circuit logique numérique qui compte chaque cycle d'horloge. Davantage de fonctionnalités sont implémentées dans le matériel pour prendre en charge le **Timer** afin qu'il puisse **compter** ou **décompter**. Il peut avoir un « **Prescaler** » pour diviser la fréquence d'horloge d'entrée par une valeur sélectionnable. Il peut également disposer de circuits pour la **capture d'entrée**, la **génération de signaux PWM** et bien plus encore :

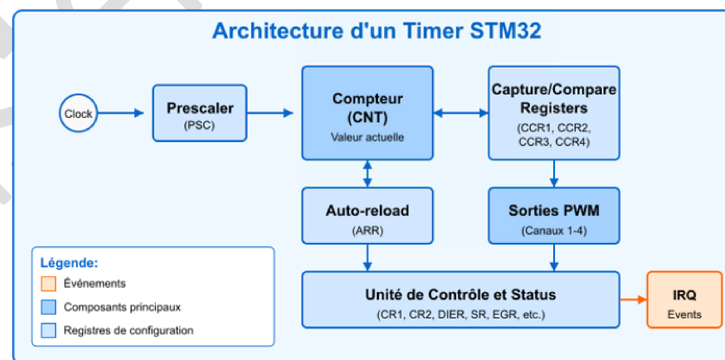


- ✓ *Capture d'entrée*
- ✓ *Comparaison de sortie*
- ✓ *Mode PWM*
- ✓ *Sortie à une impulsion*

STMicroelectronics propose différentes versions ou variantes pour les modules de minuterie matériels.

1- Modules de Timer de base

Les minuteries de base se composent d'un compteur de rechargement automatique de 16 bits (ou 32 bit) piloté par un **Prescaler** programmable. Ils peuvent être utilisés comme minuteries génériques pour la génération de bases de temps. Les composants principaux d'un **Timer** sont donnés comme suit :



- **Prescaler (PSC)** : Divise l'horloge d'entrée pour ajuster la fréquence de comptage.
- **Compteur (CNT)** : Registre qui contient la valeur actuelle du compteur
- **Auto-reload Register (ARR)** : Définit la valeur maximale du compteur

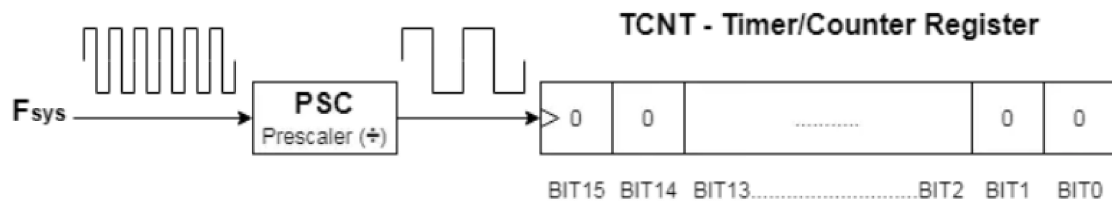
➔ **Capture/Compare Registers (CCR)** : Pour capturer des événements ou u comparer avec CNT.

Le bloc principal du Timer programmable est un compteur **16 bits** avec son registre de **rechargement** automatique associé. Le compteur peut être divisé par un **Prescaler**. Le **compteur**, le registre de **rechargement** automatique et le registre **Prescaler** peuvent être écrits ou lus par logiciel. Cela est vrai même lorsque le compteur est en marche. L'unité de base de temps comprend :

- Registre de compteur (Counter Register) (**TIMx_CNT**)
- Registre du pré-échelonneur (Prescaler Register) (**TIMx_PSC**)
- Registre de rechargement automatique (Auto-Reload Register) (**TIMx_ARR**)

Les **Timer 6 et 7** de la carte STM32F429 sont principalement utilisées pour le déclenchement DAC et la génération de formes d'onde. Ils peuvent également être utilisés comme **base de temps** générique 16 bits.

Considérons un Timer de **16 bits** de base comme celle présentée ci-dessous. Timer de **16 bits** → peut compter de **0 à 65535**. À chaque cycle d'horloge, la valeur du temporisateur est incrémentée de 1. Et comme vous pouvez le voir, le **Fsys** n'est pas la fréquence qui incrémente le module temporisateur. Mais il est divisé par le **Prescaler**, puis transmis au minuteur.



Par exemple, en mode minuterie, le registre **TCNT** est incrémenté de 1 à chaque cycle d'horloge à la fréquence suivante (**Fsys/PSC**). Cela signifie que si le **Fsys** est de **80 MHz** et le **PSC** est de **1: 1024**, le **TCNT** est incrémenté de 1 toutes les **12,8 µSec**. Par conséquent, si vous démarrez ce minuteur pour compter de 0 jusqu'à ce qu'il atteigne un débordement (overflow) (à 65535), il vous donnera un signal d'interruption toutes les 0,839 seconde.

Que se passe-t-il si on doit configurer ce Timer pour qu'elle nous donne un signal d'interruption toutes les **1 seconde** ? En fait, on ne veut pas de cet intervalle de temps de 0,839 seconde. Pour cette raison, on utilise la fonctionnalité matérielle du **registre de précharge (Preload Register)** qui force le temporisateur à compter à partir de n'importe quelle valeur arbitrairement choisie jusqu'au débordement.

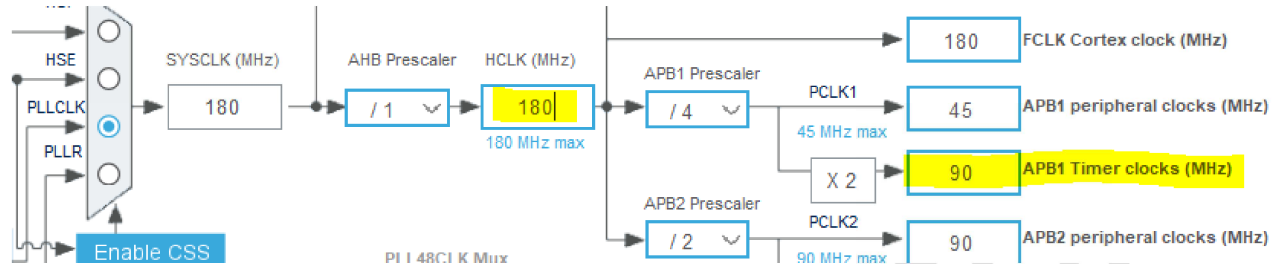
$$CNT_CLK = \frac{TIMx_CLK}{(TIMx_PSC + 1)(TIMx_ARR + 1)} \quad (1)$$

Remarque : **TIMx_CLK** représente la fréquence du Timer. Pour déterminer cette valeur, veuillez consulter la datasheet (à la fin du document) pour obtenir la fréquence de l'**APB** à laquelle le Timer est connecté.

Application :

Générez une période de 500 ms (càd $CNT_CLK = 2$) avec $TIMx_CLK = 90$ MHz, utilisez la configuration ci-dessous dans « Clock Configuration »

Quel sont les valeurs de $TIMx_PSC$ et $TIMx_ARR$ peut-on utiliser pour avoir cette période ?

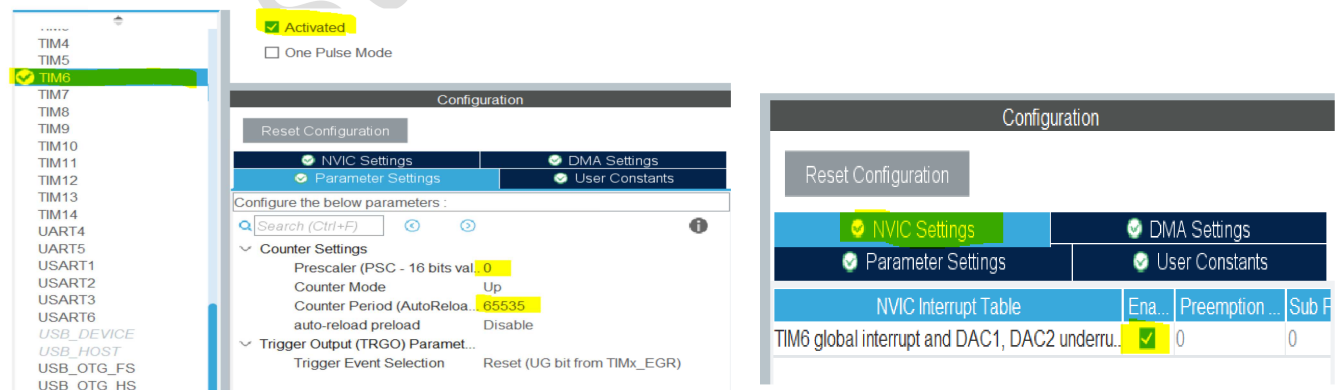


Sélectionnez **Timer6**, configurez les valeurs trouvées du **Prescaler** et du **ARR** et activer « **auto-reload Preload** » dans la configuration correspondante du Timer6 (figure ci-dessous). En appliquant la formule (1) et $TIMx_PSC = 8999$ et $TIMx_ARR = 0$, nous obtenons $CNT_CLK = 10000$ qui est une horloge de 0.1 ms. Ainsi, pour obtenir une valeur de $CNT_CLK = 2$ nous divisons 10000 par la valeur souhaitée de CNT ($TIMx_ARR + 1 = 5000$). Cela nous donne :

$$CNT_CLK = \frac{90 \times 10^6}{(8999 + 1)(4999 + 1)} = 2$$

➔ Il faut donc avoir 5000 comptes pour avoir une période de 500 ms.

- En se servant de la configuration du Timer6 comme référence temporelle, générez et ajoutez le code requis pour faire clignoter la LED à un intervalle de 500 ms. Utilisez les **interruptions** du Timer pour déclencher une interruption à chaque fin du cycle du Timer, signalant ainsi la fin des 500 ms.



Code sans interruption :

```
/* Démarre le timer */
HAL_TIM_Base_Start(&htim6);

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    if (__HAL_TIM_GET_COUNTER(&htim6) >= 4999) {
        HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_13);
    }
}
```

Code Avec interruption :

```
/* USER CODE BEGIN 2 */

// Start timer
HAL_TIM_Base_Start_IT(&htim6);
// Get current time
timer_val = __HAL_TIM_GET_COUNTER(&htim6);

/* USER CODE END 2 */
/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
    /* Vérifiez quelle version du timer a déclenché ce rappel
    et bascule la LED*/
    if (htim->Instance == htim6)
    {
        /* votre code */
    }
}
```

- 2- Écrivez une fonction nommée 'OurDelay' qui personnalise les délais en millisecondes en prenant une période générée par Timer6 égale à 1 ms. Utilisez cette fonction pour allumer une LED pendant 1 seconde puis l'éteindre pendant 2 secondes. **Remarque** : sans utiliser les interruptions.

